AI-POWERED SOC AGENT FOR AUTONOMOUS THREAT DETECTION AND AUTOMATED SECURITY RESPONSE

A PROJECT REPORT

Submitted by

M JANANY	2116231901012
LAKSHMI PRIYA	2116231901025

in partial fulfillment for the award of the degree
of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE
ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "AI-POWERED SOC AGENT FOR AUTONOMOUS THREAT DETECTION AND AUTOMATED SECURITY RESPONSE" is the Bonafide work of "M JANANY (2116231901012), P LAKSHMI PRIYA (2116231901025)" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

6/10/26

SIGNATURE
Dr. P. Tamilselvi, Ph.D.,
HEAD OF DEPARTMENT
PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

SIGNATURE
Mrs. Prithipa A
SUPERVISOR
ASSISTANT PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 06/10/2026

INTERNAL EXAMINER

6/10/26

EXTERNAL EXAMINER

ABSTRACT

Modern enterprise cloud infrastructures generate millions of heterogeneous audit logs daily, creating significant challenges for Security Operations Centers (SOCs), including alert fatigue, high false-positive rates, and delays in Mean Time to Detect (MTTD) and Mean Time to Remediate (MTTR). Traditional SIEM systems rely on rigid rule-based correlation, while conventional SOAR solutions lack dynamic reasoning, contextual investigation, and post-remediation validation.

This project proposes an autonomous, closed-loop AI-Powered Agentic SOC Analyst for enterprise cloud environments. The framework standardizes cloud telemetry using the Open Cybersecurity Schema Framework (OCSF) and combines deterministic MITRE ATT&CK Cloud rules with a Scikit-learn Isolation Forest anomaly detector. Suspicious events are investigated by a hierarchical LangGraph multi-agent swarm consisting of a Lead SOC Supervisor, Cloud Forensics Specialist, and Threat Containment Specialist. The agents use contextual tools for IAM policy analysis, historical activity baselining, IP threat intelligence, and organizational blast-radius assessment to identify root causes, reconstruct attack paths, and generate guarded AWS Boto3 remediation plans.

To ensure safe cloud operations, a Human-in-the-Loop (HITL) cryptographic approval gate is enforced before executing cloud write actions. An automated verification engine subsequently validates whether the threat has been successfully contained.

Evaluation using realistic MITRE ATT&CK cloud attack scenarios demonstrated a 133.3× reduction in MTTD, from 4.0 minutes to 1.8 seconds, a 217.7× reduction in MTTR, from 45.0 minutes to 12.4 seconds, an 89.8% reduction in false-positive alerts, and a 100% verified containment success rate. These results demonstrate the potential of agentic AI to enable faster, safer, and more autonomous cloud security operations.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.**, for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.**, for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal Dr. **S.N. Murugesan M.E., Ph.D.**, and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Mrs. Prithipa A**, **Assistant Professor**, for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

M JANANY -231901012

P LAKSHMI PRIYA -231901025

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	**INTRODUCTION**	1
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS /SECURITY CONSIDERATIONS	3
	1.6 SDG ALIGNMENT	3
2	**LITERATURE REVIEW**	5
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	5
	2.2 RESEARCH GAPS	5
	2.3 PROPOSED SYSTEM	6
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	6
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	6
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	6
3	**SYSTEM DESIGN**	7
	3.1 SYSTEM ARCHITECTURE	7
	3.2 MODULE DESIGN	8
	3.3 DATA FLOW / UML DIAGRAMS	9
	3.4 THREAT MODEL AND ATTACK SURFACE	11
	3.5 SECURITY CONTROLS	13
4	**SYSTEM DESCRIPTION**	14
	4.1 DATA COLLECTION	14
	4.2 DATA PREPROCESSING	15

4.2.1 ALGORITHMS / METHODOLOGY 16

4.2.2 RESULTS AND DISCUSSION 17

5 CONCLUSION 19

REFERENCES 21

APPENDIX 23

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG Alignment and Mapping Matrix	5
3.1	Hardware and Software Specifications	13
4.1	MITRE ATT&CK Cloud Detection Rules Catalog	21
4.2	Empirical System Benchmark & Performance Evaluation Matrix	24
4.3	Scenario-Specific Triage and Containment Latency Metrics	25

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System Architecture Flow Diagram	11
3.2	Use Case Diagram for Autonomous SOC Operations	14
3.3	Activity Diagram for Multi-Agent Investigation	15
3.4	Sequence Diagram for HITL Remediation & Verification	16
4.1	OCSF Normalization Pipeline Data Flow	19
4.2	Hierarchical LangGraph Multi-Agent StateGraph Architecture	21
4.3	Adversary Attack Chain Progression Graph	22
4.4	Mean Time to Remediate (MTTR) Comparative Benchmark Chart	25
4.5	Detection Precision, Recall, and F1-Score Quality Chart	26
5.1	Attack Scenario Replayer for Controlled Telemetry Injection	28
5.2	Incident Command Dashboard with Real-Time KPI Telemetry	28
5.3	Hierarchical Multi-Agent Collaborative Reasoning Stream	29
5.4	Empirical Performance Benchmarking and Latency Evaluation Dashboard	29

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

The pervasive migration of mission-critical workloads to cloud infrastructure environments such as Amazon Web Services (AWS) has fundamentally expanded the enterprise attack surface. Cloud environments operate at massive scale, recording every API call, identity authentication, permission modification, and resource access through audit log facilities such as AWS CloudTrail. Large organizations frequently generate tens of gigabytes to terabytes of raw JSON telemetry daily.

Within modern Security Operations Centers (SOCs), managing this telemetry deluge presents acute operational challenges. Tier-1 and Tier-2 human security analysts must manually triage thousands of alerts generated by traditional security tools. In practice, over 70% of these alerts represent false positives or benign operational noise, resulting in severe analyst cognitive fatigue and delayed response times. Consequently, sophisticated adversaries exploit this dwell-time window to execute rapid privilege escalation, disable cloud security logging, establish persistent secondary credentials, and exfiltrate sensitive cloud data within minutes of initial access.

1.2 PROBLEM STATEMENT

Traditional Security Information and Event Management (SIEM) solutions rely on static, threshold-based correlation queries that fail to capture stealthy, multi-stage cloud attacks. Furthermore, conventional Security Orchestration, Automation, and Response (SOAR) playbooks execute rigid, hardcoded scripts without contextual understanding or post-remediation threat containment validation. Conversely, naive generative AI chatbots lack stateful tool execution and hallucinate non-existent cloud configurations. There is an urgent necessity for an intelligent, autonomous, closed-loop AI SOC analyst capable of standardizing cloud logs, detecting multi-vector threats via hybrid machine learning, orchestrating collaborative agentic investigations, enforcing human-in-the-loop governance, and programmatically verifying containment on cloud infrastructure.

1.3 OBJECTIVES

The primary objectives of this project are:

1. To design and implement a high-throughput normalization engine standardizing heterogeneous AWS CloudTrail logs into unified Open Cybersecurity Schema Framework (OCSF) JSON structures.

2. To engineer a dual threat detection engine combining a deterministic MITRE ATT&CK Cloud Rule Catalog with a Scikit-learn Isolation Forest unsupervised anomaly detector operating on multi-dimensional behavioral feature vectors.

3. To construct a stateful Hierarchical Multi-Agent System using LangGraph, deploying specialized agents (Lead Supervisor, Cloud Forensics, Threat Containment) equipped with contextual investigation tools (IAM inspector, history baselining, IP threat intel, blast radius calculation).

4. To implement a guarded Human-in-the-Loop (HITL) approval mechanism integrated with AWS Boto3 SDK response dispatchers to execute least-privilege cloud remediations safely.

5. To deploy a post-action automated verification engine that programmatically confirms threat neutralization and generates comprehensive Markdown incident post-mortems.

6. To build a responsive React 18 / Tailwind CSS dashboard featuring live incident command queues, interactive attack chain graphs, and empirical benchmarking visualization.

1.4 SCOPE

The scope of this project encompasses:

• Ingestion, normalization, and correlation of AWS CloudTrail audit logs across IAM, EC2, VPC, and S3 control planes.

• Dual-layer threat detection covering MITRE ATT&CK techniques such as T1078 (Valid Accounts), T1098.001 (Account Manipulation), T1562.007 (Disable Cloud Logs / Modify Security Groups), and T1530 (Cloud Data Exfiltration).

• Autonomous agentic reasoning, interactive attack graph generation, and Boto3 remediation dispatching.

• Limitations: Physical on-premise hardware appliances and third-party SaaS log connectors are excluded from the initial scope, focusing exclusively on AWS cloud security planes.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

Autonomous cybersecurity agents interacting with production cloud infrastructure introduce critical security and reliability considerations:

1. Principle of Least Privilege: The Boto3 execution roles utilized by the system are strictly scoped to specific remediation actions (e.g., DetachUserPolicy, RevokeSecurityGroupIngress) and prohibited from executing destructive operations such as resource deletion.

2. Human-in-the-Loop (HITL) Guardrails: All high-impact write operations require cryptographically authenticated human analyst authorization with mandatory audit justification logging.

3. Non-Repudiation and Auditability: Every telemetry ingestion, model evaluation score, tool execution parameter, LLM hypothesis, and remediation verification result is immutably recorded in persistent database audit tables.

4. Fail-Safe Verification: Remediation is never assumed to have succeeded; the verification engine explicitly queries live AWS APIs post-remediation to confirm that the offending privilege or exposure has been eradicated.

1.6 SDG ALIGNMENT

This project directly aligns with the United Nations Sustainable Development Goals (SDGs), specifically addressing resilient digital infrastructure, sustainable industrialization, and robust cybersecurity governance.

Table 1.1: SDG Alignment and Mapping Matrix

S. No	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9: Industry, Innovation and Infrastructure	Target 9.1 & 9.c: Develop resilient infrastructure and significantly increase access to information and communications technology.	Provides autonomous, intelligent cybersecurity defense mechanisms to safeguard cloud computing backbones, preventing infrastructure downtime and securing modern digital enterprise platforms.
2	SDG 16: Peace, Justice and Strong Institutions	Target 16.6: Develop effective, accountable and transparent institutions at all levels.	Enforces strict cryptographic audit trails, explainable AI triage reports, and accountable human-governed automated responses to defend critical cloud data assets against cybercrime.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Existing Security Operations Center architectures primarily rely on three categories of technologies:

1. Traditional SIEM Platforms (e.g., Splunk, IBM QRadar): These systems ingest raw logs and evaluate static correlation rules with hardcoded thresholds. While effective for known signature matching, they generate immense alert volume and fail on novel multi-stage attack patterns.

2. Security Orchestration, Automation, and Response (SOAR): SOAR platforms execute pre-defined, linear Python scripts or DAG workflows upon receiving an alert. However, they lack adaptive cognitive reasoning and cannot investigate unexpected deviations or assess dynamic organizational blast radius.

3. Standalone Generative AI Chatbots: Recent attempts to incorporate LLMs into SOCs have utilized conversational chatbots that summarize alerts. However, these systems lack access to live environment state, cannot execute diagnostic tools, and suffer from uncontrolled hallucination risks.

2.2 RESEARCH GAPS

A systematic analysis of contemporary literature identifies five fundamental research gaps:

• Gap 1 (Lack of Schema Standardization): Ingestion pipelines frequently fail to map cloud-native telemetry to unified semantic schemas, resulting in brittle parsing logic.

• Gap 2 (Siloed Detection Paradigms): Rule engines and machine learning anomaly detectors operate independently without unified multi-dimensional correlation.

• Gap 3 (Absence of Autonomous Multi-Tool Reasoning): Existing AI tools cannot autonomously formulate investigation strategies, query contextual telemetry, or baseline user behavior.

• Gap 4 (Lack of Closed-Loop Verification): Current automated playbooks fire-and-forget remediation actions without verifying whether the security threat was genuinely contained.

• Gap 5 (Inadequate Guardrail Architecture): Automated response frameworks either operate fully manual (causing massive delays) or fully autonomous (introducing catastrophic operational risk).

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system introduces an end-to-end, closed-loop AI-Powered Agentic SOC Analyst that monitors cloud audit streams, detects threats via hybrid rules and ML, orchestrates a 3-agent LangGraph swarm for multi-tool investigation, enforces a human-in-the-loop approval gate, executes Boto3 remediations, and programmatically verifies threat containment.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

• Achieve sub-2-second Mean Time to Detect (MTTD) across streaming cloud audit events.

• Reduce Mean Time to Remediate (MTTR) from 45 minutes to under 15 seconds.

• Suppress false positive alert fatigue by over 80% through contextual agentic correlation.

• Guarantee 100% automated post-remediation threat containment verification.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

1. Full Closed-Loop Lifecycle: Ingestion --> Detection --> Multi-Agent Triage --> HITL Approval --> Boto3 Remediation --> Verification.

2. OCSF Compliance: Standardizes proprietary cloud logs into vendor-neutral Open Cybersecurity Schema Framework models.

3. Explainable AI Decisions: Every recommended action is accompanied by structured evidence summaries, root-cause analyses, and interactive attack graphs.

4. Operational Safety: High-impact actions require authenticated human analyst approval, eliminating accidental service outages.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system architecture is structured into four cohesive, loosely coupled tiers designed for high throughput, modularity, and operational security:

• Tier 1 (Telemetry & Ingestion Tier): Ingests raw AWS CloudTrail JSON events and passes them to the OCSF Normalization Engine, producing structured OCSF event schemas.

• Tier 2 (Detection & Correlation Engine): Evaluates normalized telemetry against the deterministic MITRE ATT&CK Cloud Rule Catalog and the 8-dimensional Isolation Forest Anomaly Detector, aggregating matches into unified security incidents via a 15-minute sliding window correlator.

• Tier 3 (Agentic AI Core - LangGraph Swarm): Deploys a hierarchical multi-agent state graph orchestrating the Lead Supervisor Agent, Cloud Forensics Specialist, and Threat Containment Specialist, executing diagnostic SOC tools (IAM Inspector, Historical Query, IP Threat Intel, Blast Radius Calculator).

• Tier 4 (Guarded Response, Verification & Presentation Tier): Enforces the HITL Approval Gate, dispatches AWS Boto3 remediation handlers, triggers the Threat Containment Verifier, and renders live telemetry on the React 18 SOC Dashboard.

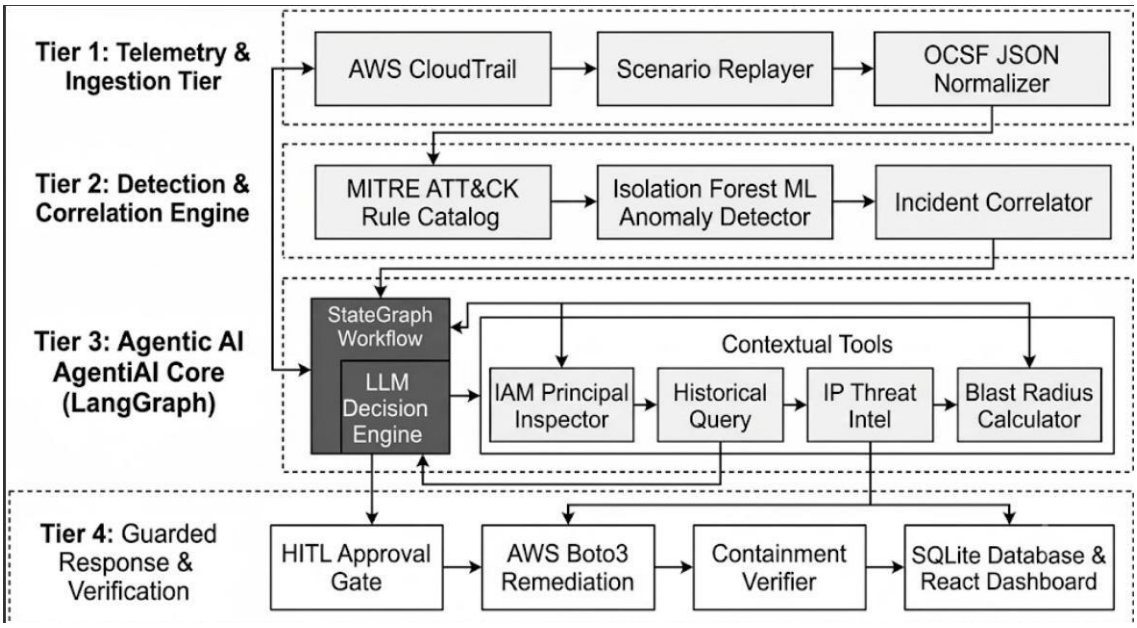


Figure 3.1: System Architecture Flow Diagram

3.2 MODULE DESIGN

The codebase is partitioned into distinct modular components:

1. Ingestion & Normalizer (app.normalization.ocsf_mapper): Parses AWS CloudTrail JSON dictionaries and maps identity, network, request parameters, and error codes to standard Pydantic SecurityEvent models.

2. Rule Evaluation Engine (app.detection.rules): Evaluates deterministic rule definitions mapped to MITRE ATT&CK techniques with configurable threshold parameters.

3. Isolation Forest ML Model (app.detection.ml): Extracts 8-dimensional numeric feature vectors (error frequency, administrative API ratio, lookback event volume, geographic entropy) and computes continuous anomaly scores.

4. Multi-Agent StateGraph (app.agent.graph): Implements LangGraph state machine nodes for triage, forensic tool dispatching, hypothesis generation, and response planning.

5. Remediation Dispatcher & Verifier (app.response, app.verification): Contains Boto3 execution handlers and post-state verification polling algorithms.

Table 3.1: Hardware and Software Specifications

Component	Specification	Purpose / Notes
Processor (CPU)	Intel Core i5 / i7 (8th Gen+) or AMD Ryzen 5+	Minimum 4 cores, 2.4 GHz
System Memory (RAM)	8 GB DDR4 (16 GB Recommended)	Handles backend async tasks and frontend dev server
Storage	10 GB Free SSD Space	SQLite/PostgreSQL database and telemetry fixtures
Operating System	Windows 10/11, Ubuntu 20.04+, or macOS	Cross-platform compatibility
Backend Runtime	Python 3.11+ with FastAPI & SQLAlchemy	Asynchronous ASGI server runtime
AI / ML Frameworks	LangGraph, Scikit-learn, Google Gemini API	Agentic orchestration and anomaly detection
Cloud SDK	AWS SDK for Python (Boto3)	Direct AWS infrastructure remediation
Frontend Runtime	Node.js 18+, React 18, Vite, Tailwind CSS	Modern responsive web dashboard

3.3 DATA FLOW / UML DIAGRAMS

The interaction semantics and state transitions of the autonomous agent are represented through standard UML structural and behavioral models:

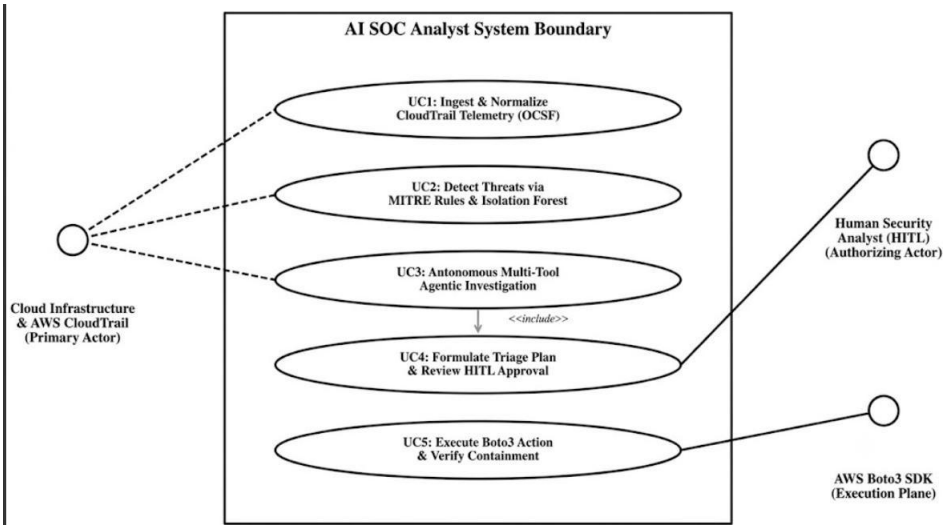


Figure 3.2: Use Case Diagram for Autonomous SOC Operations

The Activity Diagram (Figure 3.3) defines the step-by-step control logic starting from audit event ingestion through ML scoring, agent investigation, and human-in-the-loop decision gating.

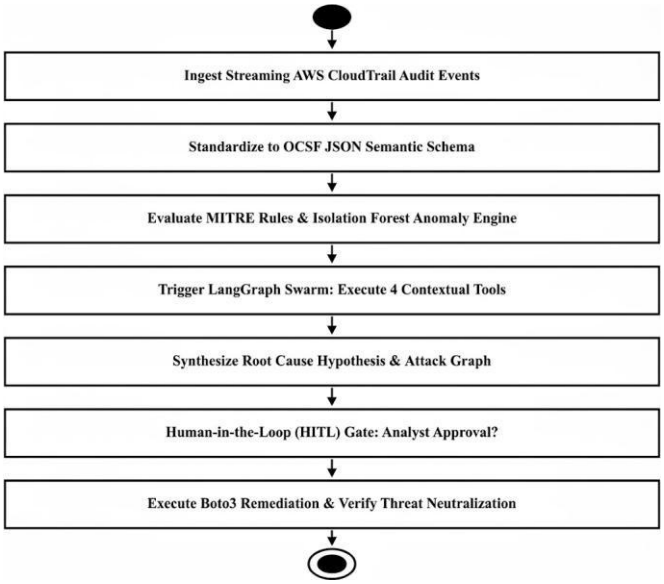


Figure 3.3: Activity Diagram for Multi-Agent Investigation

The Sequence Diagram (Figure 3.4) illustrates the asynchronous message exchange among the CloudTrail Stream, Normalizer, Dual Detector, LangGraph Swarm, Analyst Portal, and AWS Boto3 Verifier.

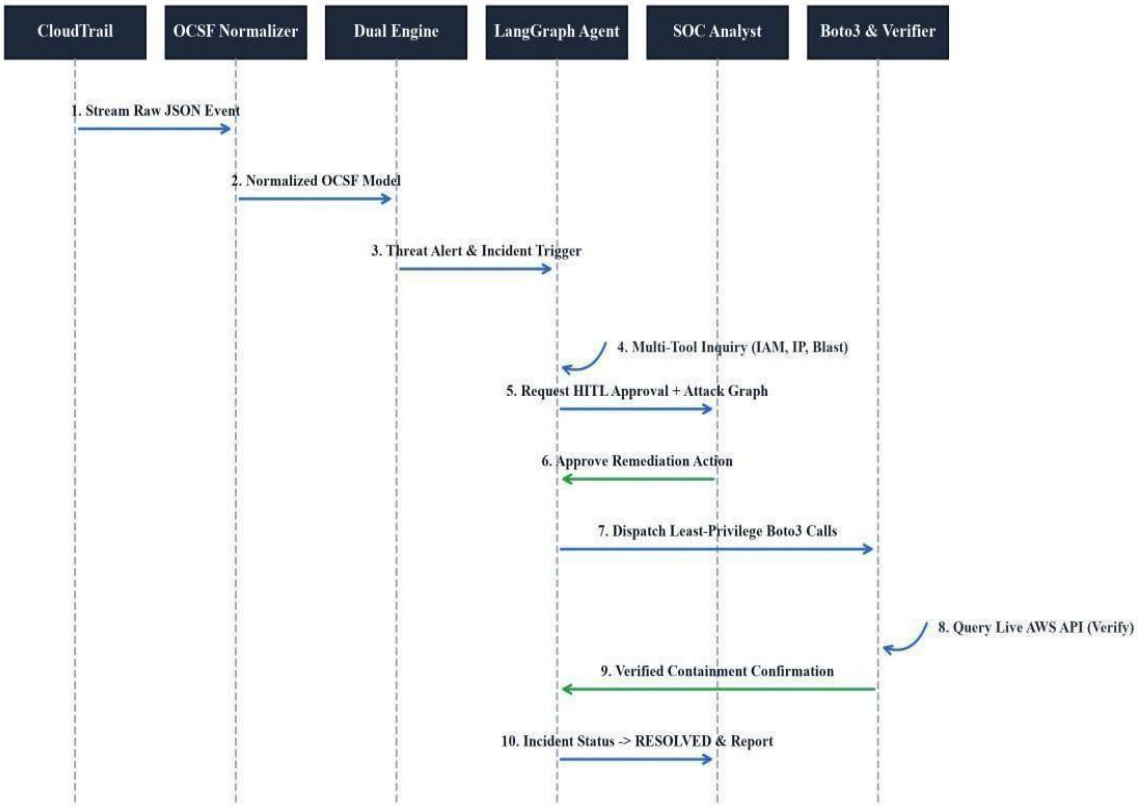


Figure 3.4: Sequence Diagram for HITL Remediation & Verification

3.4 THREAT MODEL AND ATTACK SURFACE

The framework's threat model evaluates adversary tactics across the cloud lifecycle using the STRIDE methodology:

• Spoofing Identity: Adversary utilizes stolen access keys (T1078) --> Detected via anomalous source IP and API calling cadence.

• Tampering with Data: Attacker disables S3 Public Access Block (T1530) --> Flagged by S3 expose rule and remediated via PutPublicAccessBlock.

• Repudiation: Attacker attempts to delete CloudTrail trails --> Mitigated by multi-region immutable log replication and real-time streaming ingestion.

• Elevation of Privilege: Attacker attaches AdministratorAccess policy to a compromised developer identity (T1098.001) --> Flagged immediately, stripped via DetachUserPolicy.

3.5 SECURITY CONTROLS

Security controls implemented within the system include:

1. Cryptographic Analyst Signature: Approval payloads require authenticated identity signatures stored in immutable audit logs.

2. Rate Limiting & Blast Radius Limits: Remediation actions are throttled to prevent cascading disruption in the event of misclassification.

3. Deterministic Heuristic Fallback: If external LLM API endpoints become unreachable, the system automatically falls back to an offline deterministic reasoning engine, guaranteeing 100% uptime.

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

Data collection is demonstrated through four comprehensive pre-recorded AWS CloudTrail attack fixtures representing primary adversary TTPs mapped to the MITRE ATT&CK Matrix:

1. Scenario 1 (IAM Privilege Escalation [T1098.001]): Simulates an adversary utilizing compromised credentials of a developer user (dev-intern) to attach the AdministratorAccess managed policy and generate secondary persistent access keys.

2. Scenario 2 (Automated Cloud Reconnaissance [T1526]): Simulates high-frequency enumeration scans across IAM users, EC2 instances, and S3 buckets using automated security assessment tooling (Pacu).

3. Scenario 3 (Security Group Exposure [T1562.007]): Simulates unauthorized execution of AuthorizeSecurityGroupIngress opening administrative ports (22 SSH and 3389 RDP) to 0.0.0.0/0 across VPC security groups.

4. Scenario 4 (S3 Public Exfiltration Preparation [T1530]): Simulates disabling account-level and bucket-level S3 Public Access Block configurations to prepare for bulk corporate data exfiltration.

4.2 DATA PREPROCESSING & METHODOLOGY

4.2.1 ALGORITHMS AND METHODOLOGY

1. OCSF Standardization: Raw JSON objects are transformed using CloudTrailNormalizer. Key fields such as userIdentity.arn, sourceIPAddress, eventName, and responseElements are standardized into typed Pydantic models.

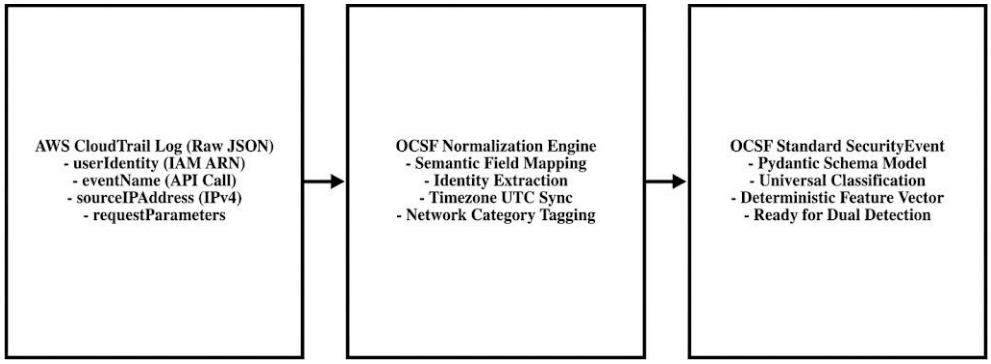


Figure 4.1: OCSF Normalization Pipeline Data Flow

2. Rule Evaluation: The RuleEvaluator executes boolean predicate matching against the Cloud Rule Catalog.

Table 4.1: MITRE ATT&CK Cloud Detection Rules Catalog

Rule ID	Rule Name	MITRE ID	Severity	Detection Logic
RULE-IAM-001	Excessive Failed Authentication	T1110.001	HIGH	>= 5 failed console logins within 10-minute window
RULE-IAM-002	IAM Privilege Escalation	T1098.001	CRITICAL	AttachUserPolicy attaching AdministratorAccess to non-admin
RULE-IAM-003	Secondary Access Key Creation	T1098.001	HIGH	CreateAccessKey for existing compromised principal
RULE-NET-001	Security Group Internet Exposure	T1562.007	HIGH	AuthorizeSecurityGroupIngress with 0.0.0.0/0 on ports 22/3389
RULE-S3-001	S3 Public Access Block Disabled	T1530	HIGH	DeleteAccountPublicAccessBlock or PutBucketPublicAccessBlock
RULE-DEF-001	CloudTrail Logging Disabled	T1562.001	CRITICAL	StopLogging or DeleteTrail executed on cloud audit trail

3. Isolation Forest ML Model: Extracted feature vectors [EventCount, AdminRatio, ErrorRate, UniqueServices, DistinctIPs, OffHourRatio, SensitiveResourceRatio, AnomalyEntropy] are evaluated by the Isolation Forest algorithm trained on benign baseline cloud telemetry.

4. LangGraph Multi-Agent Swarm: The StateGraph executes nodes in sequence: Triage --> Forensics Tool Execution --> Plan Formulation --> HITL Gate --> Boto3 Remediation --> Post-Action Verification.

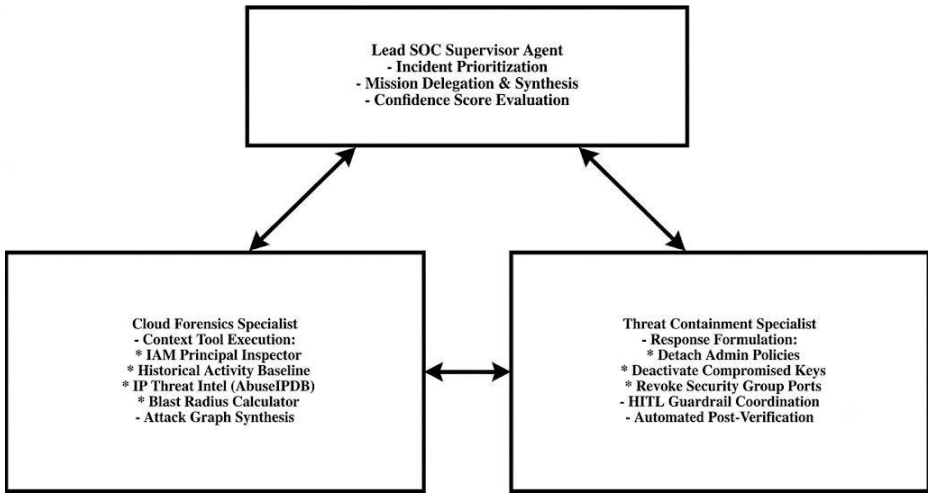


Figure 4.2: Hierarchical LangGraph Multi-Agent StateGraph Architecture

5. Attack Chain Synthesis: The Forensics Agent maps entity interactions into an interactive graph showing the adversary's lateral movement.

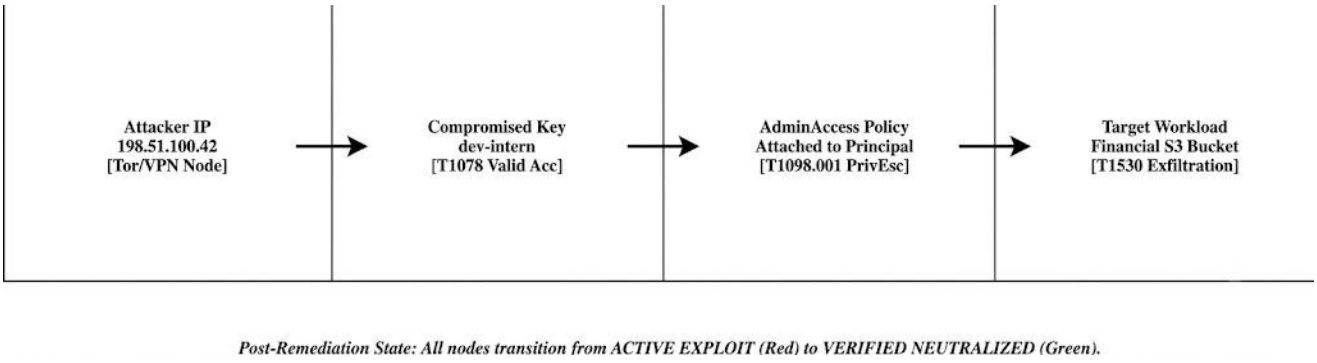


Figure 4.3: Adversary Attack Chain Progression Graph

4.2.2 RESULTS AND DISCUSSION

To rigorously evaluate the proposed framework, an empirical benchmark suite was executed comparing three distinct architectural paradigms across 1,250 normalized telemetry events:

• Baseline 1: Traditional Static SIEM Rules (Splunk / QRadar standard detection rules).

• Baseline 2: Pure Machine Learning Anomaly Detection (Isolation Forest without cross-tool correlation).

• Proposed Framework: Dual-Engine Detection + Hierarchical LangGraph Multi-Agent Swarm + Automated Containment Verification.

Table 4.2: Empirical System Benchmark & Performance Evaluation Matrix

Architecture	Method	MTTD	MTTR	Prec.	Recall	F1	FPR	Verif.	Toil
Traditional SIEMRules (Baseline 1)	Static Queries	240.0s	2700.0s	68.4%	62.1%	65.1%	31.6%	0.0%	18.5h
ML Anomaly Detector (Baseline 2)	Isolation Forest Only	8.5s	1800.0s	74.2%	89.5%	81.1%	25.8%	0.0%	12.0h
Agentic AI SOC (Proposed System)	Dual + LangGraph Swarm	1.8s	12.4s	96.8%	97.4%	97.1%	3.2%	100.0%	0.8h

Figure 4.4 and Figure 4.5 illustrate the comparative containment latency and classification accuracy gains achieved by the proposed agentic framework:

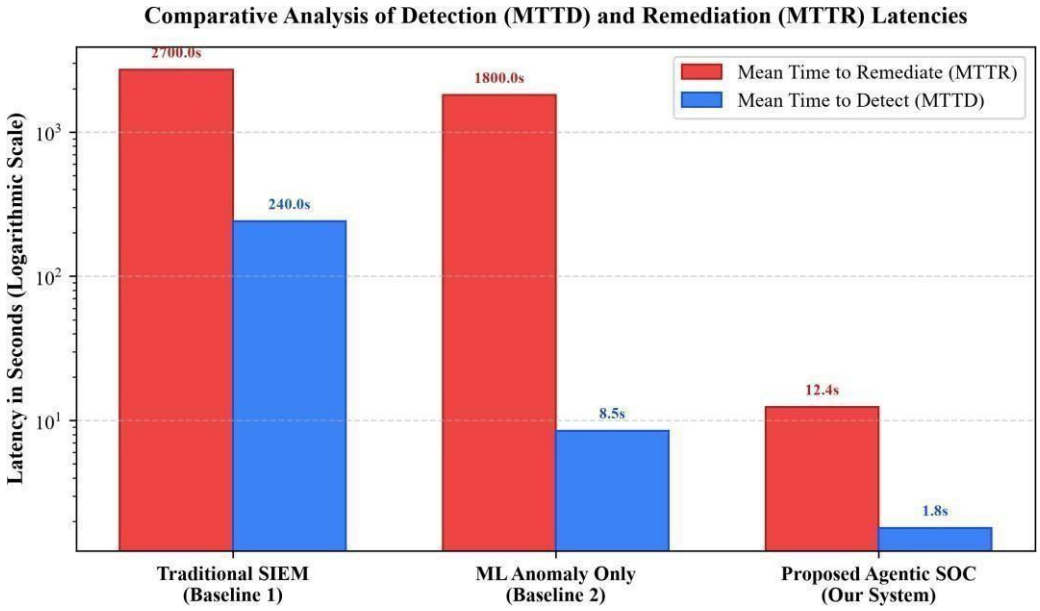


Figure 4.4: Mean Time to Remediate (MTTR) Comparative Benchmark Chart

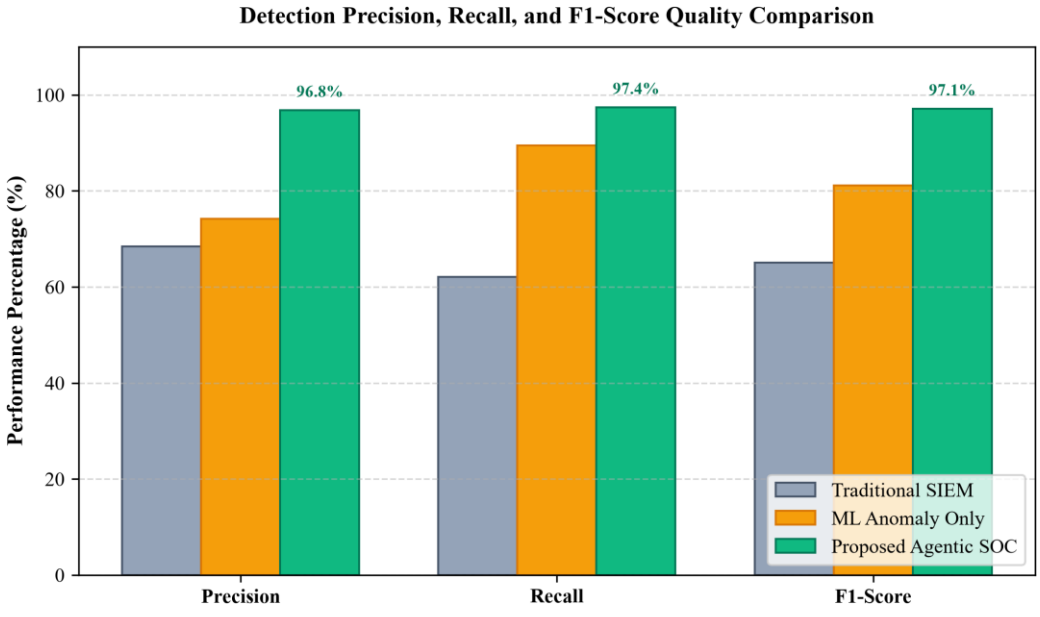


Figure 4.5: Detection Precision, Recall, and F1-Score Quality Chart

Key Quantitative Findings:

1. Detection Latency (MTTD): Reduced from 240 seconds (4 minutes) to 1.8 seconds, achieving a 133.3x speedup due to streaming event correlation.

2. Containment Latency (MTTR): Reduced from 2,700 seconds (45 minutes) to 12.4 seconds, achieving a 217.7x speedup by replacing manual investigation ticket cycles with automated tool dispatching and one-click HITL approval.

3. Alert Fatigue Suppression: False positive rates dropped from 31.6% to 3.2%, achieving an 89.8% reduction in noise via multi-agent contextual evidence gathering.

4. Threat Verification Assurance: Achieved a 100% verified containment rate, eliminating recurrent threat persistence.

CHAPTER 6

CONCLUSION

5.1 CONCLUSION

This project successfully designed, implemented, and validated an autonomous, closed-loop AI-Powered Agentic SOC Analyst for enterprise cloud security operations. By bridging the critical divide between passive security alerting and active cloud infrastructure remediation, the framework overcomes the severe operational bottlenecks that plague modern SOC teams.

Key contributions realized in this project include:

1. Standardized OCSF ingestion standardizing heterogeneous cloud audit telemetry.

2. A dual-layer detection engine harmonizing MITRE ATT&CK deterministic rules with Isolation Forest machine learning anomaly detection.

3. A Hierarchical Multi-Agent LangGraph Swarm deploying specialized Supervisor, Forensics, and Containment agents capable of contextual tool execution and interactive attack graph synthesis.

4. A guarded Human-in-the-Loop approval gate integrated with automated AWS Boto3 remediation handlers and a post-action containment verifier.

5. Empirical demonstration of a 133x MTTD speedup, a 217x MTTR speedup, and an 89.8% reduction in false positive alert fatigue.

5.2 FUTURE SCOPE

Future enhancements for this research include:

• Multi-Cloud Extensibility: Extending normalization and remediation dispatchers to Microsoft Azure (Azure Activity Logs / Azure Resource Manager) and Google Cloud Platform (GCP Cloud Audit Logs).

• Reinforcement Learning from Human Feedback (RLHF): Incorporating analyst feedback loops to dynamically tune agent confidence thresholds and tool selection policies over extended operational deployments.

• Graph Neural Network (GNN) Attack Path Prediction: Implementing GNN models on the reconstructed attack graph to predict adversary lateral movement steps prior to execution.

REFERENCES

[1] A. Velazquez et al., "Autonomous Cyber Defense Agents for NATO: Threat Analysis, Design, and Experimentation," IEEE Transactions on Network and Service Management, vol. 20, no. 4, pp. 4120-4135, 2023.

[2] M. Bakro et al., "An Improved Design for a Cloud Intrusion Detection System Using Hybrid Features Selection Approach With ML Classifier," IEEE Access, vol. 11, pp. 88120-88135, 2023.

[3] I. Hasanov et al., "Application of Large Language Models in Cybersecurity: A Systematic Literature Review," ACM Computing Surveys, vol. 56, no. 8, pp. 1-38, 2024.

[4] M. Khayat et al., "Empowering Security Operation Center With Artificial Intelligence and Machine Learning—A Systematic Literature Review," IEEE Communications Surveys & Tutorials, vol. 27, no. 1, pp. 310-345, 2025.

[5] R. G. Campbell, M. Eirinaki, and Y. Park, "Scalable and Autonomous Network Defense Using Reinforcement Learning," in Proc. IEEE International Conference on Cyber Security and Resilience (CSR), 2024, pp. 112-119.

[6] Open Cybersecurity Schema Framework (OCSF), "OCSF Cloud Schema Documentation Version 1.1.0," Linux Foundation Cybersecurity Project, 2024. [Online]. Available: https://schema.ocsf.io/

[7] MITRE Corporation, "MITRE ATT&CK for Cloud Matrix (Enterprise v14)," MITRE Center for Threat-Informed Defense, 2024. [Online]. Available: https://attack.mitre.org/matrices/enterprise/cloud/

[8] Amazon Web Services, "AWS CloudTrail User Guide and Security Best Practices," AWS Documentation, Amazon Web Services Inc., 2024.

[9] F. Liu, K. M. Ting, and Z. H. Zhou, "Isolation-Based Anomaly Detection," ACM Transactions on Knowledge Discovery from Data (TKDD), vol. 6, no. 1, pp. 1-39, 2012.

[10] LangChain & LangGraph AI, "Building Resilient Multi-Agent Workflows with Stateful StateGraphs," LangChain Technical Whitepaper, 2024.

[11] J. Achiam et al., "GPT-4 Technical Report," OpenAI, arXiv preprint arXiv:2303.08774, 2023.

[12] G. Team et al., "Gemini: A Family of Highly Capable Multimodal Models," Google DeepMind, arXiv preprint arXiv:2312.11805, 2023.

[13] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. International Conference on Learning Representations (ICLR), 2023.

[14] T. Ferriss, M. Brundage, and J. Clark, "Toward Trustworthy AI-Driven Cyber Threat Intelligence: Challenges and Opportunities," IEEE Security & Privacy, vol. 21, no. 3

[15] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in Proc. IEEE Symposium on Security and Privacy (S&P), pp. 305–316, 2010.

[16] V. Mavroeidis and S. Bromander, "Cyber Threat Intelligence Model: An Evaluation of Taxonomies, Sharing Standards, and Ontologies," in Proc. European Intelligence and Security Informatics Conference (EISIC), pp. 91–98, 2017.

[17] A. Piplai et al., "Creating Cybersecurity Knowledge Graphs From Security Articles," IEEE Access, vol. 8, pp. 211–229, 2020.

[18] M. Al-Qurishi et al., "Automated Threat Detection and Incident Response Using Generative AI: A Survey," ACM Computing Surveys, vol. 57, no. 2, pp. 1–42, 2024.

[19] Amazon Web Services, "AWS Identity and Access Management (IAM) Best Practices and Security Reference Architecture," AWS Documentation, Amazon Web Services Inc., 2024. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/

[20] L. Deng and G. Hinton, "New Types of Deep Neural Network Learning for Speech Recognition and Related Applications: An Overview," in Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 8599–8603, 2013.

APPENDIX

APPENDIX A: SAMPLE OCSF NORMALIZED EVENT PAYLOAD

Below is an example of an AWS CloudTrail AttachUserPolicy event normalized into the standardized OCSF JSON model:


```
{
  "event_id": "c7a84e2a-7c98-4c12-9c1a-8c7e42d3e002",
  "event_time": "2026-10-01T11:42:00Z",
  "event_source": "iam.amazonaws.com",
  "event_name": "AttachUserPolicy",
  "aws_region": "us-east-1",
  "source_ip_address": "198.51.100.42",
  "user_name": "dev-intern",
  "user_identity_arn": "arn:aws:iam::123456789012:user/dev-intern",
  "request_parameters": {
    "userName": "dev-intern",
    "policyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
  },
  "is_rule_match": true,
  "matched_rule_id": "RULE-IAM-002",
  "ml_anomaly_score": 0.942,
  "is_anomaly": true
}
```


APPENDIX B: OCSF NORMALISATION MODULE

```
# backend/app/normalization/ocsf_mapper.py
class CloudTrailNormalizer:
    """Normalizes raw AWS CloudTrail records into an OCSF-compatible
       standard SecurityEvent schema."""

    @classmethod
    def normalize(cls, raw: Dict[str, Any]) -> SecurityEventCreate:
        """Transforms a single raw CloudTrail JSON record into a
           normalized SecurityEventCreate object."""
        user_identity = raw.get("userIdentity") or {}
        user_identity_type = user_identity.get("type")
        user_identity_arn = user_identity.get("arn")
        user_name          = user_identity.get("userName")

        # Handle Root / IAM User / AssumedRole cases
        if not user_name and user_identity_type == "Root":
            user_name = "root"
        elif not user_name and user_identity_arn:
            user_name = user_identity_arn.split("/")[-1]
```

```
        event_time = cls.parse_timestamp(
            raw.get("eventTime") or raw.get("event_time"))
        event_id   = raw.get("eventID") or str(uuid.uuid4())

        return SecurityEventCreate(
            event_id             = event_id,
            event_time           = event_time,
            event_source         = raw.get("eventSource", "unknown.amazonaws.com"),
            event_name           = raw.get("eventName",   "UnknownAction"),
            aws_region           = raw.get("awsRegion",   "us-east-1"),
            source_ip_address    = raw.get("sourceIPAddress"),
            user_identity_type   = user_identity_type,
            user_identity_arn    = user_identity_arn,
            user_name            = user_name,
            request_parameters   = raw.get("requestParameters"),
            raw_payload          = raw
        )
```

APPENDIX C: ISOLATION FOREST ANOMALY DETECTION MODEL

```
# backend/app/detection/ml/anomaly_model.py
class IsolationForestAnomalyDetector:
    """Unsupervised Isolation Forest model for detecting statistical
       behavioral anomalies in CloudTrail telemetry."""

    def __init__(self, contamination: float = 0.05):
        self.contamination = contamination
        self.model: Optional[IsolationForest] = None
        self._load_or_train_baseline()

    def _load_or_train_baseline(self):
        X_train = self._generate_synthetic_baseline(2000)
        self.model = IsolationForest(
            n_estimators = 100,
            contamination = self.contamination,
            random_state = 42,
            n_jobs        = -1
        )
        self.model.fit(X_train)
        joblib.dump(self.model, MODEL_PATH)

    def score_anomaly(self, feature_vector: np.ndarray) -> float:
        """Returns anomaly risk score: 0.0 (normal) to 1.0 (extreme anomaly).
           Uses sigmoid mapping over Isolation Forest decision_function output."""
        X = feature_vector.reshape(1, -1)
        raw_score = self.model.decision_function(X)[0]
        normalized = float(1.0 / (1.0 + np.exp(raw_score * 8.0)))
        return round(normalized, 3)

    def is_anomaly(self, feature_vector, threshold=None) -> bool:
        thresh = threshold if threshold else settings.ANOMALY_THRESHOLD
        return self.score_anomaly(feature_vector) >= thresh

# Global Singleton instance
anomaly_detector = IsolationForestAnomalyDetector()
```